AI-Powered Video Hosting Platform

Complete Startup Blueprint & Technical Roadmap

"Video Infrastructure + AI Video Understanding — Powered by Meta SAM2"

Document Type	Startup Roadmap & Technical Blueprint
Platform	STRAVIA — Video Hosting + AI
Core Technology	Meta SAM2, FFmpeg, HLS, CDN
Target Founder	Solo Full-Stack Developer
Stack	Next.js · Python · FFmpeg · PyTorch · AWS/GCP
Version	1.0 — 2025 Edition

This document covers 11 comprehensive sections — from startup vision and market analysis to full technical architecture, development roadmap, and business strategy. Built for solo founders ready to start coding immediately.

SECTION 01

What is STRAVIA?

STRAVIA is a next-generation, AI-native video hosting platform designed to compete with and surpass platforms like Vimeo, Mux, and Gumlet. At its core, STRAVIA handles the full video lifecycle: ingestion, compression, encoding into streaming formats (HLS/DASH), cloud storage, and CDN delivery through a polished embeddable player. What makes STRAVIA fundamentally different is a deep AI layer powered by Meta's Segment Anything Model 2 (SAM2) — enabling the platform to understand, index, and interact with video content at the object and frame level.

The Core Problem

Video content is the fastest-growing format on the internet, yet the tools available to developers and creators remain fundamentally static. Existing video hosting platforms treat video as a passive binary blob — upload it, compress it, play it back. There is no concept of 'what is inside the video.' This creates several painful limitations:

- Creators cannot search for specific moments, objects, or scenes within their video library.
- Developers cannot build interactive video experiences without expensive custom engineering.
- Brands spend hours manually creating clips and thumbnails that AI could generate automatically.
- Moderation is either manual (expensive) or keyword-based (inaccurate).
- Metadata is limited to title, description, and duration — no semantic understanding.

Why Traditional Platforms Fall Short

Vimeo, Wistia, Mux, and Cloudflare Stream are excellent engineering products, but they were designed in a pre-AI era. Their architectures are optimized for throughput and delivery — not understanding. Adding AI capabilities retroactively to their infrastructure is architecturally difficult and commercially unattractive for them, as it would cannibalize their simplicity advantages. This creates a greenfield opportunity for a platform built AI-first from day one.

Why SAM2 is the Right Differentiator

Meta's Segment Anything Model 2 (SAM2) represents a generational leap in computer vision. It can segment and track arbitrary objects across video frames in real time, with zero task-specific training required. This means STRAVIA can:

- Detect and track any object a user points to inside a video
- Generate structured metadata (bounding boxes, masks, labels) for every frame
- Enable semantic search — 'find all videos where a red car appears'
- Create interactive hotspot overlays on any object in any video
- Auto-generate smart clips around detected events or objects
- Produce AI thumbnails by identifying visually compelling frames
- Power content moderation by detecting prohibited objects or scenes

The Vision

STRAVIA's long-term vision is to become the infrastructure layer for intelligent video on the web — the way Stripe became infrastructure for payments and Twilio became infrastructure for communications. Every developer building a video product should be able to plug into STRAVIA's API and instantly get not just fast video delivery, but deep AI understanding of every second of every video they host. STRAVIA will make video searchable, interactive, and

intelligent by default.

SECTION 02

The video hosting and streaming infrastructure market is worth over $10 billion and growing at approximately 20% annually, driven by the explosion of video content across enterprise, education, media, and developer tooling use cases. Here is a detailed breakdown of the key players.

Vimeo

The OG creator-focused video platform. Built for filmmakers, agencies, and enterprise video teams. Offers clean embeds, privacy controls, and a strong video player. Strengths: Brand trust, beautiful player, creator community, OTT tools. Weaknesses: Expensive for high-volume usage, no developer API for serious infra work, no AI features, storage limits feel punishing. Falling behind in the developer market.

Mux

The developer-first video API. Mux abstracts all video infrastructure behind clean REST APIs. Strengths: Excellent DX, data analytics, real-time video, predictive QoE metrics. Weaknesses: Pure infrastructure play — no AI, no search, no object understanding. Usage-based pricing gets expensive at scale. Strong technical moat but narrow focus.

Gumlet

A lean video hosting platform targeting SMBs and startups. Strengths: Affordable, fast setup, good player, image + video CDN combo. Weaknesses: Thin feature set, no AI capabilities, limited analytics, smaller engineering team limits roadmap velocity.

Cloudflare Stream

Cloudflare's video hosting product built on top of their global edge network. Strengths: Incredible CDN, extremely competitive pricing, near-zero latency globally. Weaknesses: Very basic product — no analytics depth, no AI, no advanced player. Intended as a commodity product, not a platform.

Wistia

The B2B/marketing video platform. Focuses on lead generation, video SEO, and viewer analytics. Strengths: Best-in-class video analytics, CRM integrations, gated video/lead capture. Weaknesses: Expensive, slow to innovate, no developer API, no AI features. Strong PMF in marketing but weak everywhere else.

Bunny Stream

Budget video hosting with global CDN. Extremely price-competitive. Strengths: Low cost, fast CDN, straightforward API. Weaknesses: Bare-minimum features, no analytics, no AI, customer support is limited. Pure commodity play.

The Opportunity Gap

Every player in this market is competing on the same axis: upload speed, CDN performance, and price. None of them are competing on video intelligence. The gap is clear: there is no video hosting platform today that can answer the question 'What is happening inside this video?' STRAVIA is built to answer that question.

SECTION 03

The table below provides a structured comparison of key competitors across the dimensions most relevant to STRAVIA's positioning.

Company	Primary Focus	Target Customers	Key Strengths	Key Weaknesses
Vimeo	Creator hosting	Filmmakers, agencies, enterprise	Brand trust, beautiful player, community	No AI, expensive at volume, limited API
Mux	Video API / infra	Developers, startups	Developer UX, analytics, real-time	No AI/search, pricy at scale, narrow scope
Gumlet	SMB hosting	Small teams, startups	Affordable, fast setup, image+video	Thin features, no AI, limited scale
Cloudflare Stream	Edge delivery	Developers, cost-conscious teams	Global CDN, very cheap, zero latency	No analytics, no AI, commodity product
Wistia	Marketing video	B2B marketers, sales teams	Video analytics, lead capture, CRM	Expensive, slow roadmap, no AI, no API
Bunny Stream	Budget hosting	Cost-sensitive developers	Lowest cost, fast CDN, simple API	Bare features, no analytics, no AI
STRAVIA	**AI video infra**	**Developers, creators, enterprise**	**SAM2 AI, object search, smart clips**	**New entrant, needs traction**

Where STRAVIA Fits

STRAVIA occupies a unique position at the intersection of reliable video infrastructure and cutting-edge AI video understanding. Unlike Mux (infra only) or Vimeo (consumer focus), STRAVIA targets developers and product teams building the next generation of video-powered applications — teams who need both the reliability of a Mux-style API and the intelligence of an AI vision layer. The initial beachhead market is B2B SaaS companies, e-learning platforms, and media agencies who need searchable, interactive video at scale.

SECTION 04

MVP — Phase 1

Video Upload	Resumable chunked uploads up to 10GB. Support MP4, MOV, AVI, MKV. Show upload progress. Handle failures gracefully with retry logic.
Video Compression	Automatic compression using FFmpeg. Reduce file size by 60-80% while maintaining visual quality. CRF-based encoding for optimal size/quality ratio.
HLS Encoding	Encode every video into HLS with adaptive bitrate profiles: 360p, 720p, 1080p. Generate .m3u8 playlist and .ts segments automatically.
Cloud Storage	Store all encoded files and segments in S3-compatible storage (AWS S3 or Cloudflare R2). Organized by workspace and video ID.
Video Streaming	Custom HLS player built on HLS.js. Auto bitrate switching. Buffering intelligence. Keyboard shortcuts. Mobile-friendly.
Embeddable Player	Generate embed codes (iframe + JS snippet). Custom branding. Privacy controls. Domain restrictions.
Dashboard	Upload interface, video library, basic stats (plays, duration viewed).
API	REST API for programmatic upload, retrieval, and deletion. API key auth.

Phase 2 — Enhanced Platform

Video Analytics	Play rate, watch time heatmaps, drop-off points, unique viewers, geographic data, device breakdown. Real-time dashboard.
Thumbnail Generation	Auto-generate 5 thumbnail candidates using FFmpeg frame extraction. Allow user to select or upload custom thumbnail.
Chapter Markers	Allow creators to add timestamped chapters. Display in player UI.
Collections & Folders	Organize videos into projects and folders. Team access controls.
Webhook Events	Fire webhooks on upload complete, encoding done, video deleted.
Subtitle/Caption Support	Upload SRT/VTT files. Auto-generate captions using Whisper AI.
Multi-quality Downloads	Allow controlled download with quality selection.

Phase 3 — AI Features (SAM2)

Object Detection	Run SAM2 on every uploaded video. Detect and segment all prominent objects. Store bounding boxes + masks per frame in database.
Object Tracking	Track detected objects across all frames. Build temporal object trajectories. Expose tracking data via API.
AI Search	Full-text search across AI-generated metadata. Query: 'find videos with a laptop on a desk' — returns timestamped results.
Smart Thumbnails	Use SAM2 to identify visually rich frames with prominent objects. Auto-select the best thumbnail based on object clarity score.
Automatic Clips	Detect scene changes and object activity spikes. Auto-generate short highlight clips with SAM2 guidance.
Interactive Hotspots	Allow creators to place clickable overlays on tracked objects. Click on a product in a video → open link. Powered by SAM2 masks.
AI Moderation	Detect and flag prohibited objects, nudity indicators, or violent content. Human review queue with AI pre-screening.
AI Metadata API	Expose structured JSON metadata per video: objects detected, timestamps, confidence scores, labels.

Future Features

Multi-language Transcription	Whisper-powered transcription in 90+ languages.
AI Video Editor	Natural language video editing: 'Remove all silent pauses' or 'Cut the clip to 60 seconds keeping the key moments.'
Live Streaming	RTMP ingest, transcoding to HLS, global distribution.
DRM / Encryption	Widevine + FairPlay DRM for premium content protection.
AI Chaptering	Automatically detect topic changes and generate chapters.
Video Templates	Branded intro/outro templates applied automatically.
Multi-CDN Failover	Route traffic through multiple CDNs for 99.99% availability.

SECTION 05

Below is a comprehensive catalogue of technologies available for each layer of a video hosting platform. Multiple options are provided in each category so the founder can make informed trade-offs.

Frontend

Next.js 14+	Full-stack React framework. SSR, SSG, API routes. Best choice for dashboard + marketing site.
React	Pure client-side SPA. Good for dashboard only. More setup needed for SSR.
Vue.js / Nuxt	Lighter alternative to React. Good DX. Smaller ecosystem.
SvelteKit	Excellent performance. Smaller bundle. Steeper learning curve from React.
TailwindCSS	Utility-first CSS. Fast to build with. Works with any framework.
HLS.js	JavaScript library for HLS playback in browsers. Industry standard.
Video.js	Full-featured open-source video player. Highly customizable.
Shaka Player	Google's adaptive streaming player. Excellent for DASH + DRM.

Backend

Node.js + Express	Lightweight, familiar for JS devs. Good for API and upload handling.
Node.js + Fastify	Faster than Express. Better TypeScript support. Growing adoption.
Python + FastAPI	Required for AI processing layer. Async, typed, OpenAPI built-in.
Go (Golang)	Excellent for high-throughput upload services. Compiled performance.
Rust (Actix)	Maximum performance for video processing coordination. Steep learning curve.
BullMQ (Node.js)	Redis-backed job queue for video processing tasks.
Celery (Python)	Distributed task queue for Python. Used for SAM2 processing workers.

Video Processing

FFmpeg	The gold standard. Handles transcoding, compression, HLS packaging, thumbnail extraction. Free, open-source.
GStreamer	Lower-level pipeline toolkit. More flexible than FFmpeg. Steeper learning curve.
Shaka Packager	Google's tool for HLS/DASH packaging, encryption, and DRM.
HandBrake CLI	User-friendly transcoder built on FFmpeg. Good for presets.
GPU Encoding (NVENC)	NVIDIA hardware encoder. 10x faster than CPU encoding. Requires GPU servers.
Intel Quick Sync	Intel hardware encoding. Lower cost than NVIDIA. Good quality.
AWS MediaConvert	Managed video transcoding service. No server management. Pay per minute.
Cloudflare Stream	Upload and they handle encoding. Very simple. Less control.

Encoding Formats

HLS (HTTP Live Streaming)	Apple standard. Best browser + mobile support. Used by Netflix, YouTube.
MPEG-DASH	Open standard. Better for DRM. Supported by Shaka Player.
WebM / VP9	Open format. Smaller files. Limited Safari support.
H.264 (AVC)	Universal codec. Best compatibility. Larger files than H.265.
H.265 (HEVC)	50% smaller than H.264. Less browser support. Patent licensing.
AV1	Next-gen open codec. Smallest files. Slow encoding. Growing support.
ABR (Adaptive Bitrate)	Multiple quality levels in one stream. Auto-switches based on bandwidth.

Storage

AWS S3	Industry standard object storage. Reliable, global, cheap at scale. Best ecosystem.
Cloudflare R2	S3-compatible, zero egress fees. Massive cost savings for high-bandwidth apps.
Google Cloud Storage	Strong ML integration. Good for teams using GCP/Vertex AI.
Backblaze B2	Cheapest S3-compatible storage. Good for archival tiers.
MinIO	Self-hosted S3-compatible storage. Full control. Good for on-prem hybrid.
Wasabi	Low-cost, no egress fees. Slower than S3. Good for archive storage.

CDN / Streaming

Cloudflare CDN	Free tier, 300+ PoPs globally, DDoS protection built-in. Best price/performance.
AWS CloudFront	Deep S3 integration. Mature, feature-rich. More expensive.
Bunny CDN	Cheap, fast, 120+ PoPs. Simple pricing. Great for video streaming.
Fastly	Edge compute, real-time purging, enterprise grade. More expensive.
KeyCDN	Simple, fast, pay-as-you-go. Good for MVPs.
Akamai	Enterprise CDN. Largest network. Very expensive. For post-Series-B scale.

AI / Machine Learning

Meta SAM2	Core AI differentiator. Object segmentation + tracking in video. PyTorch-based.
PyTorch	Deep learning framework for running SAM2 and other models.
CUDA	NVIDIA GPU computing platform. Required for fast SAM2 inference.
OpenCV	Computer vision library for frame extraction and preprocessing.
Hugging Face Transformers	Access to thousands of pre-trained models. Easy model deployment.
OpenAI Whisper	State-of-the-art speech recognition. For auto-captions and transcription.
CLIP (OpenAI)	Image-text embeddings. Enables natural language video search.
YOLO (v8/v10)	Fast object detection. Lightweight alternative to SAM2 for detection only.

TensorRT	NVIDIA inference optimization. 3-5x faster model serving.
ONNX Runtime	Cross-platform model inference. Deploy PyTorch models efficiently.

Database

PostgreSQL	Best relational DB for SaaS. JSONB for metadata. Full-text search. Reliable.
MySQL / PlanetScale	Alternative to Postgres. PlanetScale adds serverless scaling.
MongoDB	Document store. Flexible schema for AI metadata. Good for unstructured data.
Redis	In-memory cache + pub/sub. Used for job queues, session cache, rate limiting.
Pinecone	Vector database for AI embeddings. Enables semantic video search.
Qdrant	Open-source vector DB. Self-host alternative to Pinecone.
ClickHouse	Columnar DB for analytics. Handles billions of play events efficiently.
TimescaleDB	Time-series Postgres extension. Good for video analytics data.

Queues / Workers

BullMQ	Redis-backed job queue for Node.js. Supports priorities, retries, cron.
Celery	Python task queue. Used for SAM2 inference workers. Supports Redis/RabbitMQ.
RabbitMQ	Message broker. Reliable message delivery between services.
AWS SQS	Managed message queue. Integrates well with Lambda and EC2 workers.
Temporal	Durable workflow orchestration. Handles complex multi-step pipelines.
Apache Kafka	High-throughput event streaming. For real-time analytics pipeline.

Infrastructure / Cloud

AWS (EC2, S3, ECS, Lambda)	Most mature cloud. Largest service ecosystem. Standard choice.
Google Cloud Platform	Best for ML workloads. Strong GPU availability. Vertex AI integration.
Cloudflare Workers + R2	Zero-cost egress. Edge compute. Excellent for video distribution.
Railway / Render	Simple PaaS for MVP deployment. No DevOps knowledge required.
Fly.io	Docker-based global deployment. Fast to set up. Good for API services.
Hetzner	Cheapest GPU servers in EU. 80% cheaper than AWS for compute-heavy workloads.
Lambda GPU	Dedicated GPU cloud. A100/H100 availability. ML-optimized instances.
Docker + Kubernetes	Container orchestration. Needed at scale. Overkill for MVP.
Terraform	Infrastructure as code. Reproducible deployments across cloud providers.

Monitoring & Observability

Sentry	Error tracking and performance monitoring. 5-minute setup.

Datadog	Full observability platform. APM, logs, metrics, dashboards.
Grafana + Prometheus	Open-source metrics stack. Self-hosted. Free but requires setup.
Posthog	Product analytics + session recording. Great for understanding user behavior.
LogTail / Logtail	Modern log management. Cheaper than Datadog for early stage.
Uptime Robot	Free uptime monitoring with alerting. Good for MVP monitoring.

Authentication

Clerk	Developer-first auth. UI components + API. Fast to integrate. 10k MAU free.
Auth0	Enterprise-grade auth. More complex, more expensive. Good for B2B.
NextAuth.js	Open-source auth for Next.js. Free. Requires more configuration.
Supabase Auth	Built-in auth if using Supabase. Row-level security integration.
Lucia	Lightweight TypeScript auth library. Full control, no vendor lock-in.

Payments

Stripe	Industry standard. Subscriptions, usage-based billing, invoicing. Best DX.
Paddle	Full merchant of record. Handles global taxes automatically. Good for SaaS.
LemonSqueezy	Simpler Paddle alternative. Good for indie hackers and early-stage.
Chargebee	Enterprise subscription management. Connects to Stripe/Braintree.

SECTION 06

From all the options evaluated, the following stack is recommended for a solo founder building STRAVIA. Every choice optimizes for development speed, cost efficiency, scalability, and developer experience simultaneously.

Frontend	**Next.js 14 + TypeScript + TailwindCSS**
	Next.js gives you a full-stack framework in one — API routes, SSR, SSG, and the dashboard all in one codebase. TypeScript catches bugs early. Tailwind is the fastest way to build good-looking UIs without writing CSS. The ecosystem is huge and hiring is easy later.
Video Player	**HLS.js (custom player)**
	HLS.js is the industry standard for HLS playback in browsers. Build a thin React wrapper around it for full customization. This avoids video.js bloat while giving you complete control over the player UI and behavior.
Backend API	**Node.js + Fastify (TypeScript)**
	Fastify is 2x faster than Express with better TypeScript support and built-in schema validation. Keep the main API in TypeScript to share types with the frontend. This handles uploads, auth, video metadata, and the dashboard API.
AI Processing	**Python + FastAPI + PyTorch**
	SAM2 runs on Python/PyTorch — there's no way around this. FastAPI is the cleanest Python web framework with async support and automatic OpenAPI docs. Run this as a separate microservice that the Node.js backend calls for AI tasks.
Video Processing	**FFmpeg (self-hosted workers)**
	FFmpeg is the undisputed king of video processing. Run FFmpeg workers on dedicated EC2/Hetzner instances. Use GPU instances (NVENC) for faster encoding — 10x speedup over CPU encoding. Keep workers stateless so they can scale horizontally.
Job Queue	**BullMQ + Redis**
	BullMQ handles the async video processing pipeline perfectly. Jobs are queued when uploads complete and workers pick them up. Supports retries, priorities, and cron jobs. Redis also handles session caching and rate limiting for free.
Storage	**Cloudflare R2**
	R2 is S3-compatible with ZERO egress fees. For a video platform this is enormous — you'll save thousands per month compared to AWS S3 at scale. Same API as S3 so switching is trivial if needed. Store all video segments, thumbnails, and processed assets here.

CDN	**Cloudflare CDN**
	Pair R2 with Cloudflare CDN for global video delivery. Cloudflare has 300+ PoPs worldwide, built-in DDoS protection, and aggressive caching for video segments. The R2 + CDN combo is the most cost-effective video delivery stack available.
Database	**PostgreSQL (Neon or Supabase)**
	Postgres handles relational data (users, videos, workspaces, subscriptions) and JSONB columns for AI metadata. Use Neon for serverless Postgres or Supabase for the bundled auth + storage + realtime features. Start managed, optimize later.
Vector Search	**pgvector (Postgres extension)**
	For AI-powered video search using CLIP embeddings, pgvector adds vector search directly to Postgres. Avoid running a separate vector DB (Pinecone) until you genuinely need it — pgvector handles millions of vectors comfortably.
Authentication	**Clerk**
	Clerk provides beautiful auth UI components, social login, MFA, and organization management out of the box. The free tier supports 10,000 MAU. For a B2B SaaS this is the fastest path to production-grade auth.
Payments	**Stripe**
	Stripe is the only reasonable choice. Usage-based billing (for bandwidth/storage), subscription plans, and invoicing are all handled. Stripe's developer experience and webhook reliability are unmatched.
Monitoring	**Sentry + Posthog**
	Sentry for error tracking and performance monitoring. Posthog for product analytics and feature flags. Both have generous free tiers. Add Grafana + Prometheus for infrastructure metrics when you're serving real traffic.
Deployment	**Railway (MVP) → AWS ECS + Hetzner (Scale)**
	Start on Railway for zero-DevOps deployment — push to GitHub, it deploys. When you need GPU servers for SAM2 inference, move AI workers to Hetzner (80% cheaper than AWS). Main API and frontend stay on Railway/Vercel. Video workers move to AWS ECS.

SECTION 07

Standard Video Processing Pipeline

The video pipeline is the heart of STRAVIA. Every video goes through a deterministic multi-stage pipeline from upload to playable stream. Here is the complete flow:

01

Client Upload

The user selects a video file in the dashboard. The frontend breaks it into 5MB chunks using the File API. Each chunk is uploaded to the backend via POST /upload/chunk. The backend uses tus protocol or a custom chunked upload handler. On the final chunk, the upload is marked complete and a processing job is queued.

02

Temporary Storage

Raw uploaded chunks are assembled and stored temporarily in /tmp or an ephemeral S3 prefix. The assembled raw file is validated (check codec, duration, resolution) using FFprobe. If validation fails, the job is marked failed and the user is notified.

03

Job Queue

A BullMQ job is created with the video ID and raw file path. The job includes metadata: target resolutions, priority level, workspace settings. Workers poll the queue and pick up jobs. Multiple workers can process videos in parallel.

04

Compression

The video worker runs FFmpeg to compress the video:
ffmpeg -i input.mp4 -c:v libx264 -crf 23 -preset fast -c:a aac -b:a 128k output.mp4
CRF 23 is the sweet spot — roughly 60-70% size reduction with minimal quality loss. The preset 'fast' balances encoding speed vs compression efficiency.

05

HLS Encoding

FFmpeg encodes the compressed video into HLS with 3 quality profiles:
360p @ 400kbps, 720p @ 1500kbps, 1080p @ 4000kbps.
Output: master.m3u8 playlist + quality variant playlists + .ts segment files (6s each). Segment duration of 6 seconds is the industry standard for adaptive bitrate switching.

06

Thumbnail Generation

FFmpeg extracts frames at regular intervals (every 10% of duration). Additionally, the worker extracts the frame at the 5-second mark as a default thumbnail. All thumbnail candidates are stored in R2 under /thumbnails/{videoId}/.

07 **Upload to R2/S3**

All encoded files are uploaded to Cloudflare R2:
/videos/{workspaceId}/{videoId}/master.m3u8
/videos/{workspaceId}/{videoId}/360p/playlist.m3u8 + segments
/videos/{workspaceId}/{videoId}/720p/playlist.m3u8 + segments
/videos/{workspaceId}/{videoId}/1080p/playlist.m3u8 + segments
/thumbnails/{workspaceId}/{videoId}/thumb_00.jpg ... thumb_04.jpg

08 **CDN Distribution**

Cloudflare CDN sits in front of R2. The video player requests master.m3u8 through the CDN URL. Segments are cached at edge nodes globally. First viewer in a region fetches from R2; subsequent viewers are served from cache. Cache-Control headers set to max-age=86400 for segments (immutable) and 60s for playlists.

09 **Database Update**

The video record in Postgres is updated: status=ready, duration, resolution, file_size, hls_url, thumbnail_url, processed_at. A webhook event is fired (video.processing.complete) to notify the customer's application.

10 **Playback**

The HLS.js-powered player fetches master.m3u8 from the CDN URL. It reads variant playlists and selects the appropriate quality based on available bandwidth. The player auto-switches between quality levels as network conditions change. Buffering is handled automatically by HLS.js.

SAM2 AI Processing Pipeline

The SAM2 pipeline runs in parallel with or after the standard encoding pipeline. It is computationally expensive and should be queued as a lower-priority background job.

Frame Extraction	FFmpeg extracts 1 frame per second (or adaptive based on duration). For a 10-minute video, this yields ~600 frames. Frames are saved as JPEG to /tmp/frames/{videoId}/.
SAM2 Inference	The Python SAM2 worker loads the model (sam2_hiera_large.pt recommended). For each frame batch, SAM2 runs automatic mask generation — detecting all prominent objects without any prompts. Output: list of masks with bounding boxes and confidence scores per frame.
Object Tracking	SAM2's video predictor tracks detected objects across frames. Given a first-frame mask, SAM2 propagates it forward/backward through the video. This creates object trajectories: [{frameIndex, bbox, mask, confidence}].
Label Generation	OpenAI CLIP or a BLIP-2 model assigns text labels to each detected mask. Example: mask of object in region [x,y,w,h] → 'laptop', 'coffee cup', 'person'. Labels are stored alongside bounding boxes in Postgres JSONB.

Embedding Generation	CLIP generates vector embeddings for each detected object patch. Embeddings are stored in pgvector. This enables semantic search: 'find videos containing a red car' → CLIP embeds the query → nearest-neighbor search.
Metadata Storage	All SAM2 output is stored in Postgres: video_objects table: videoId, label, firstFrame, lastFrame, trajectory (JSONB) video_frames table: videoId, frameIndex, objects (JSONB array of detections)
AI Features Unlocked	Once SAM2 metadata is stored, all AI features are available: object search, interactive hotspots, smart thumbnails (highest-confidence-object frame), automatic clips (frames with most activity), AI moderation (flag prohibited objects).

SECTION 08

STRAVIA's architecture is a microservices-lite design — not a complex K8s cluster, but a set of clearly separated services communicating via API and a job queue. Here is how the full system is wired together.

Frontend (Next.js)

> Dashboard UI: video library, upload interface, analytics, settings

> Marketing site: landing page, pricing, docs (same repo, different route)

> Embeddable player: script tag or iframe that loads from player.stravia.io

> Communicates with Backend API via HTTPS REST or tRPC

> Auth state managed by Clerk, JWT passed with every API request

Upload Service

> Dedicated endpoint for chunked video uploads (POST /upload/chunk)

> Accepts chunks, writes to ephemeral storage, assembles on final chunk

> Validates assembled file with FFprobe (codec, duration, size check)

> Creates database record: status=uploading → status=processing

> Enqueues video processing job in BullMQ with video metadata

> Returns uploadId and CDN playback URL immediately for optimistic UI

Video Processing Workers

> Node.js workers listening on BullMQ 'video-encoding' queue

> Spawns FFmpeg subprocess for compression and HLS encoding

> Generates thumbnail candidates via FFmpeg frame extraction

> Uploads all outputs to Cloudflare R2 using AWS SDK (S3-compatible)

> Updates database record: status=ready, updates all URLs and metadata

> Fires webhook events on completion or failure

> Horizontally scalable: spin up more workers during peak load

AI Processing Workers (Python)

> Python workers listening on Celery 'sam2-processing' queue

> Runs on GPU-enabled instances (Hetzner or AWS g4dn instances)

> Loads SAM2 model once on startup (model stays resident in GPU memory)

> Processes frame batches asynchronously — does not block the main pipeline

> Writes AI metadata (object detections, trajectories) to Postgres

> Generates CLIP embeddings and stores in pgvector for semantic search

> Separate scaling policy: scale out only during AI processing demand spikes

Backend API (Fastify)

> REST API for all client operations: CRUD videos, workspaces, analytics

> Validates requests with Zod schemas, handles auth via Clerk JWT verification

> Proxy layer for AI features: routes SAM2 queries to Python AI service

> Webhook delivery: stores webhook configs, delivers events with retries

> Rate limiting via Redis: per API key and per IP

> Stripe webhook handler: updates subscription status, usage records

Database (PostgreSQL)

> Tables: users, workspaces, videos, video_objects, video_frames, analytics_events

> JSONB columns for flexible AI metadata storage per video

> pgvector extension for CLIP embedding storage and cosine similarity search

> Connection pooling via PgBouncer or Prisma Accelerate

> Read replicas for analytics queries to avoid impacting write performance

Cache Layer (Redis)

> Session cache: user session data for fast auth middleware

> Rate limit counters: per API key request counts

> BullMQ job queue: pending and active encoding jobs

> Celery broker: SAM2 processing task queue

> Hot video metadata cache: frequently accessed video records

Storage (Cloudflare R2)

> Buckets: stravia-videos (HLS segments + playlists), stravia-thumbnails, stravia-raw (temp)

> Lifecycle policy: delete raw uploads after 48 hours post-processing

> Access: presigned URLs for uploads, public URLs via CDN for playback

> Versioning disabled (segments are immutable, no need for versioning)

CDN (Cloudflare)

> Sits in front of R2 bucket — all video URLs go through CDN

> Cache rules: segments cached for 24h, playlists cached for 60s

> Custom domain: stream.stravia.io routes through Cloudflare

> DDoS protection and bot mitigation built-in at no extra cost

> Hotlink protection: restrict video access to authorized domains only

SECTION 09

This is a realistic 6-phase roadmap for a solo developer. Each phase builds on the last and produces shippable, testable software. Time estimates assume ~20-30 hours/week.

Foundation

Estimated Duration: 2-3 Weeks

■ Set up monorepo	Use Turborepo or pnpm workspaces. Packages: apps/web, apps/api, apps/worker.
■ Next.js dashboard shell	Auth with Clerk. Basic layout, sidebar nav, protected routes.
■ Fastify API skeleton	REST endpoints, Zod validation, Clerk JWT middleware, health check.
■ PostgreSQL schema	Users, workspaces, videos tables. Prisma ORM. Run first migration.
■ Redis setup	Local Redis via Docker. BullMQ queue setup. Test job enqueue/dequeue.
■ CI/CD pipeline	GitHub Actions: lint, typecheck, test on push. Deploy to Railway on merge to main.
■ Environment management	dotenv-vault or Doppler for secrets management across environments.

Upload & Processing Pipeline

Estimated Duration: 3-4 Weeks

■ Chunked upload UI	Uppy.js or custom implementation. Show progress bar. Handle pause/resume.
■ Upload API endpoint	POST /upload/chunk. Assemble chunks server-side. Validate with FFprobe.
■ FFmpeg worker	Node.js worker process. Spawn FFmpeg. Handle stdout/stderr. Update job progress.
■ HLS encoding	3-profile HLS output (360p, 720p, 1080p). Generate master.m3u8 and segments.
■ R2 upload	Stream encoded files to Cloudflare R2. Use multipart upload for large HLS batches.
■ Thumbnail generation	Extract 5 frames via FFmpeg. Upload to R2. Store URLs in DB.
■ Processing status API	WebSocket or polling endpoint for real-time processing status in UI.
■ Video library UI	Grid/list view of uploaded videos. Status badges. Basic video card with thumbnail.

Streaming Player

Estimated Duration: 2-3 Weeks

■ HLS.js player component	React wrapper around HLS.js. Attach to a video element. Handle events.
■ Adaptive bitrate	Let HLS.js handle ABR automatically. Add quality selector for manual override.
■ Custom player UI	Play/pause, seek bar with preview thumbnails, volume, fullscreen, speed control.
■ Embed code generator	Generate iframe and script embeds. Preview in dashboard. Copy to clipboard.
■ Privacy controls	Domain allowlist for embeds. Password protection option. Unlisted vs public.
■ CDN integration	Point HLS URLs through Cloudflare CDN. Test cache headers. Verify edge caching.
■ Mobile player testing	Test HLS playback on iOS Safari, Android Chrome. Handle native video fallback.

Analytics & Monetization

Estimated Duration: 2-3 Weeks

■ Play event tracking	Track play, pause, seek, ended, quality_change events via POST /analytics/event.
■ Analytics dashboard	Play rate, watch time, drop-off heatmap, unique viewers chart (Recharts).
■ Stripe integration	Products, prices, checkout session, customer portal, webhook handler.
■ Usage metering	Track storage used and bandwidth consumed per workspace. Report to Stripe.
■ Subscription gating	Enforce plan limits: max videos, max storage, max monthly plays.
■ API key management	Generate, rotate, revoke API keys. Per-key usage tracking.

SAM2 AI Integration

Estimated Duration: 4-6 Weeks

■ Python AI service	FastAPI app. SAM2 model loading. Celery worker. Docker container.
■ Frame extraction job	After encoding completes, queue SAM2 frame extraction job.
■ SAM2 auto segmentation	Run automatic mask generator on sampled frames. Parse output masks.
■ Object tracking	Propagate masks across video using SAM2 video predictor.
■ CLIP labeling	Send object patches to CLIP. Store text labels with detections in DB.
■ pgvector embeddings	Generate and store CLIP embeddings. Build semantic search endpoint.
■ AI search UI	Search bar in dashboard: query → vector search → timestamped results with thumbnails.
■ Interactive hotspots	UI to place clickable overlays on tracked objects. Hotspot editor.

■ **Smart thumbnails UI**	Show AI-recommended thumbnails based on SAM2 confidence scores.
■ **Auto clip generation**	Detect high-activity segments. Generate and store short clips automatically.

Scale & Polish

Estimated Duration: Ongoing

■ **GPU worker fleet**	Move SAM2 workers to Hetzner GPU instances. Auto-scale via queue depth.
■ **Multi-CDN setup**	Add Bunny CDN as fallback. Implement smart routing based on viewer location.
■ **Video analytics v2**	Real-time analytics via Kafka + ClickHouse. Handle billions of events.
■ **Enterprise features**	SSO (SAML/OIDC), audit logs, custom domains, SLA agreements.
■ **Live streaming**	RTMP ingest endpoint, real-time transcoding, HLS live output.
■ **Developer docs**	Docusaurus or Mintlify. Full API reference, guides, code examples.
■ **SDKs**	JavaScript SDK (npm package), Python SDK, webhook library.

SECTION 10

STRAVIA's business model combines SaaS subscription plans with usage-based pricing components — a proven model for developer infrastructure products (similar to Twilio, Cloudflare, and Mux). This hybrid approach captures both predictable recurring revenue and upside from high-usage customers.

Pricing Tiers

Starter	Pro	Business	Enterprise
$29/mo	$99/mo	$299/mo	Custom
✓ 25 GB storage included	✓ 100 GB storage included	✓ 500 GB storage included	✓ Unlimited storage (billed per GB)
✓ 100 GB bandwidth/month	✓ 500 GB bandwidth/month	✓ 2 TB bandwidth/month	✓ Unlimited bandwidth (billed per GB)
✓ Up to 100 videos	✓ Unlimited videos	✓ Unlimited videos	✓ Custom SLA (99.99% uptime)
✓ 720p max encoding	✓ 1080p + 4K encoding	✓ Priority encoding queue	✓ SSO / SAML integration
✓ Basic analytics	✓ Advanced analytics with heatmaps	✓ Full analytics suite	✓ Dedicated AI processing capacity
✓ Embeddable player	✓ Custom player branding	✓ SAM2 AI — 50 hrs/month	✓ Private CDN configuration
✓ API access	✓ Webhooks	✓ Interactive hotspots	✓ Custom domain streaming
✓ No AI features	✓ SAM2 AI processing — 10 hrs/month	✓ AI moderation	✓ Dedicated support + onboarding
	✓ Object search in videos	✓ Multi-workspace	✓ Custom contract & invoicing
	✓ Smart thumbnails & auto clips	✓ Priority email support	

Usage-Based Pricing (Overages)

Resource	Included in Plan	Overage Rate	Notes
Storage	Per-plan allocation	$0.02 / GB / month	S3-class pricing passed through
Bandwidth	Per-plan allocation	$0.05 / GB	Egress from CDN
AI Processing (SAM2)	Per-plan hours	$0.80 / hour	GPU compute cost + margin
API Calls	Unlimited (rate limited)	$0.0001 per call above 1M/mo	For high-volume API users

Transcoding Minutes	Included up to plan limit	$0.005 / minute	For 1080p; 4K at 3x rate

Revenue Projections (Conservative)

Based on comparable developer infrastructure products, a realistic revenue trajectory assuming focused outbound and content marketing:

Milestone	Customers	Avg MRR/Customer	MRR	ARR
Month 3 (Beta)	10	$49	$490	**$5,880**
Month 6	50	$79	$3,950	**$47,400**
Month 12	200	$99	$19,800	**$237,600**
Month 18	500	$120	$60,000	**$720,000**
Month 24	1,200	$140	$168,000	**$2,016,000**

SECTION 11

Building STRAVIA as a solo founder is entirely achievable — but only if you resist the temptation to over-engineer early. Here is a practical, honest playbook.

Validate Before You Build

> Do NOT start coding the SAM2 pipeline on day one. Start with the question: who will pay me $99/month for video hosting? Go find 10 of those people first.

> Post in developer communities (Indie Hackers, Reddit r/SaaS, Twitter/X): 'Building AI-powered video hosting. What's your biggest pain with Vimeo/Mux?' Read every response carefully.

> Create a Notion landing page describing STRAVIA. Add a waitlist. Run $50 of Google/Twitter ads. If 100 people sign up, you have signal. If 3 sign up, pivot the positioning.

> Interview 10 potential customers over Zoom. Ask: How much video content do you host? What platform do you use? What would make you switch? What would you pay?

> Find your beachhead: e-learning creators, marketing agencies, or SaaS companies with product demo videos. One specific persona first.

Build the Boring Core First

> The sexy feature is SAM2 AI. The necessary feature is reliable video hosting. Build boring first. If your HLS encoding pipeline is flaky, no AI feature will save you.

> MVP is: upload video → encode to HLS → stream from CDN → embed anywhere. That is it. Ship that and charge for it.

> Use managed services aggressively at the start. AWS MediaConvert instead of self-hosting FFmpeg workers. Cloudflare Stream instead of building your own CDN setup. Replace managed services with self-hosted as you understand usage patterns.

> Your first infrastructure decision should optimize for speed of learning, not cost efficiency. Cost efficiency matters at $10k MRR. It does not matter at $500 MRR.

Getting First Customers

> Target developers who already have a video hosting problem. They are on Vimeo, paying $20+/month, and frustrated by limits. Your job is to be a better product.

> Write content: 'How I built an HLS video pipeline in Next.js' or 'Why we moved from Vimeo to our own video infrastructure.' SEO compounds over time.

> Offer a generous free tier or unlimited free trial for the first 3 months. Feedback from 10 paying users is worth more than $300/month in MRR at this stage.

> Go to niche communities. e-learning builders on Circle. Shopify app developers. Course creators on Twitter. Offer to migrate them from Vimeo for free.

> Cold outreach to agencies: 'I see you're using Vimeo for client videos. I've built STRAVIA with AI features they don't have. Can I show you for 20 minutes?'

Avoid These Common Mistakes

> Do not build a Kubernetes cluster for your MVP. Start on Railway. Move to ECS when you have 100 paying customers and understand your traffic patterns.

> Do not build SAM2 processing before you have paying customers using basic video hosting. You will waste 6 weeks on a feature no one has validated they want.

> Do not build a multi-tenant enterprise feature set (SSO, audit logs, SAML) until an enterprise customer asks for it with a purchase order in hand.

> Do not underprice. Charging $9/month attracts low-quality customers with high support burden. $99/month filters for serious users.

> Do not build mobile apps, browser extensions, or integrations until the core web product has product-market fit.

Solo Founder Technical Leverage

> Use Cursor AI or GitHub Copilot for code generation. A solo dev with AI assistance can ship 2-3x faster than a solo dev without it.

> Keep your infrastructure boring. Postgres, Redis, S3, BullMQ, Node.js — these technologies have solved every problem you'll encounter at the answers you'll find online.

> Write integration tests for the video pipeline from day one. A broken HLS encode discovered by a customer is worse than a slow shipping velocity.

> Automate deployments from day one. Every commit to main should auto-deploy to staging. Prod deploy should be one CLI command or button click.

> Create a status page (statuspage.io free tier) and monitor video processing latency. When a customer reports an issue, you want to know before they do.

12-Month North Star Goals

> Month 1-2: Working video pipeline. Upload → encode → stream. 5 beta users.

> Month 3: First paying customer. Charge real money. Get feedback.

> Month 4-6: 25 paying customers. Basic analytics. Stable infrastructure.

> Month 6-8: SAM2 integration. Object search working. 3 customers using AI features.

> Month 9-12: 100+ paying customers. $10k MRR. Clear product-market fit signal.

> Month 12+: Hire first engineer. Focus on sales. Build enterprise tier.

Final Note from the Blueprint

STRAVIA is a product with real competitive advantage and a clear market need. The technical foundation is achievable by a single experienced developer in 3-6 months. The SAM2 AI layer provides a genuine and defensible differentiator that no current player in the market has deployed at the product level.

The founder's job in Year 1 is not to build the most technically impressive product. It is to find 100 customers who will pay for it. Everything else is secondary.

Start with the pipeline. Ship fast. Talk to customers. Iterate. Then add the AI.